Variables and Naming

Study guide | Python 3 | Spark-free | Databricks-ready examples

1. Introduction

A variable is a name bound to an object; assignment does not declare a fixed storage type. Python names are case-sensitive. Data engineering code should use descriptive names that reveal units and meaning, especially for counts, timestamps, and monetary amounts.

2. Syntax

Assign using `name = expression`; unpack related values with tuple unpacking. Constants are conventionally written in `UPPER_SNAKE_CASE`, though Python does not enforce immutability. Avoid reusing a name for a different concept midway through a transformation.

```
SOURCE_SYSTEM = "orders_api"
raw_amount = "125.50"
order_count, rejected_count = 24, 1
print(SOURCE_SYSTEM, order_count, rejected_count)
```

3. Real-world data example

A pipeline can keep `source_path`, `batch_started_at`, `accepted_count`, and `rejected_count` as distinct names rather than ambiguous names such as `x`, `data`, or `value`.

4. Various ways of using it

- Use local variables for short-lived intermediate values.
- Use function parameters and return values to make data flow explicit.
- Use configuration objects or function arguments instead of scattered magic constants.

5. Best practices

- Use `snake_case` for variables and functions; choose names that communicate units.
- Use `Decimal`-named variables for monetary values and `_count` for counts.
- Avoid shadowing built-ins such as `list`, `dict`, and `sum`.

6. Do's

- Do initialize a name before reading it.
- Do use immutable values for fixed configuration where practical.

7. Don'ts

- Don't rely on implicit global variables shared between notebook cells.
- Don't name a variable `id` if it obscures a domain identifier or shadows a useful built-in.

8. Pros

- Dynamic assignment makes small transformations concise.
- Clear naming makes data flow and review easier.

9. Cons

- A name can be rebound to an unrelated value, making code hard to reason about.

- Misspelled names can fail only when that execution path runs.

10. Practice examples with solutions

Try each task before reviewing its solution. Each example uses local, synthetic data and requires no external credentials.

Practice 1: Rename vague fields into meaningful variables for a batch size and failed-row count.

Solution

```
batch_size = 500
failed_row_count = 3
print(batch_size, failed_row_count)
```

Practice 2: Unpack a source name and row count and create a status message.

Solution

```
source_name, row_count = "customers", 120
print(f"{source_name}: {row_count} rows")
```

Practice 3: Calculate a remaining-row count without overwriting the original total.

Solution

```
total_rows, rejected_rows = 120, 4
accepted_rows = total_rows - rejected_rows
print(accepted_rows)
```

11. Databricks notes

Import the companion `.py` file into Databricks as a source notebook. The demonstrations are Spark-free and use embedded synthetic data. pandas examples require pandas in the selected runtime; standard Databricks runtimes commonly include it. Confirm runtime versions and policies before using optional packages in production.